

AUDITING PROPOSALS, PRECONDITIONS, AND OUTCOMES IN STATEFUL LLM AGENTS

Author and affiliation pending

Development manuscript — empirical evidence synthesis

ABSTRACT

An agent can complete a task safely while relying on a tool to reject a stale conditional update. Conversely, an observed rejection need not prevent a business error. We separate the requested mutation, its submitted preconditions, the committed state transition, and the eventual outcome. Two recorded native-tool diagnostics motivate the distinction. In a mutable ToolSandbox extension, both prompting conditions complete 14 of 16 cases safely; three of four failing primary episodes nevertheless perform post-write reads. In a separate SQLite extension, four prompting conditions each complete 12 of 12 cases, using 70 update attempts, of which 22 are rejected. An exhaustive retrospective audit preserves every saved prestate and business payload while retaining different subsets of the originally submitted preconditions. Eight condition-dependent payloads would affect the wrong current target without selection checks, whereas 14 rejected minimal updates remain business-safe without their failed row check. Importantly, a correctly rejected conditional request is not thereby a model error. These local checks do not estimate an unguarded agent’s performance. The evidence supports a capability-aware accounting of proposal, enforcement, and recovery, but neither prompt superiority nor a new concurrency mechanism.

Index Terms— LLM agents, tool-use evaluation, conditional updates, mutable state, reproducibility

1. INTRODUCTION AND RELATION TO PRIOR WORK

An agent asked to update a service’s current configuration can read a routing table, obtain a configuration identifier, and later submit a mutation. If the route changes without changing the old configuration row, the row’s version is insufficient to validate the current target. Alternatively, a different field of that row can change while the agent’s minimal patch remains appropriate. Both cases can trigger different relationships between a stale observation, a conditional attempt, and a business error.

Reporting only final success hides these relationships. Yet counting every rejected attempt as an error is equally misleading: an optimistic conditional update can be a legitimate

way to discover a conflict and then replan. The full action is “apply this mutation if these conditions hold,” not its mutation stripped of conditions. We therefore audit requested effects and operational preconditions separately, without treating counterfactual guard removal as the model’s actual behavior.

This distinction does not establish a new protection mechanism. HTTP defines conditional requests [1]; AgentSpec implements runtime enforcement [2]. *Mind the Gap* studies agent check/use races and mitigations [3]. *Reason Less, Verify More* studies silent wrong-state failures, pre-execution gates, and heterogeneous gate precision [4]. ATR explicitly separates version changes from decision-invalidating changes [5]. These works prevent us from claiming that revalidation, gate auditing, or semantic conflict handling is new.

We instead provide a limited, auditable empirical synthesis: (i) recorded post-write checks that confirm the wrong binding; (ii) successful conditional workflows whose effects depend on tool enforcement; and (iii) an exhaustive local replay of every recorded SQLite update, not only rejected updates. We preserve negative prompting results and distinguish lawful conditional attempts from unsafe counterfactual payloads. The remaining contribution question is whether this accounting yields additional insight on independent public tasks, beyond established enforcement analyses.

2. ACCOUNTING FOR ACTIONS AND LOCAL EFFECTS

2.1. An action includes its preconditions

Let w_i be the actual state immediately before a recorded update. The proposed action is $a_i = (m_i, p_i)$, with business mutation m_i and the set p_i of optional predicates actually supplied. A simplified conditional executor is

$$T(w_i, a_i) = \begin{cases} U(w_i, m_i), & p_i(w_i) = 1, \\ w_i, & p_i(w_i) = 0, \end{cases} \quad (1)$$

where U is the documented mutation and a failed predicate returns a visible conflict. Ordinary schema and existence checks remain in both branches. A task specifies a current-selection or literal-ID target and the business fields permitted to change. Metadata versions are not business fields.

We keep four recorded outcomes: a mutation tool returned success; the final goal holds; the final business state is clean; and the goal holds with no unauthorized write anywhere in the trace. Termination and completion statements are separate. External updates change only their logged cells; they cannot legitimize earlier agent side effects. We do not rewrite earlier endpoints to obtain a desired ranking.

For auditing, define B_i to indicate that $U(w_i, m_i)$ makes an unauthorized business change under the task’s frozen specification. B_i is a property of an explicitly modified, guard-free transition. It does not label a_i itself an illegal request, nor diagnose what the model intended. A correctly handled conflict can be normal optimistic control flow. Similarly, $p_i(w_i) = 0$ with $B_i = 0$ is a locally conservative rejection, not proof that the predicate should be removed from production.

2.2. What a same-prestate replay identifies

For each saved proposal we retain one of four subsets of p_i : none, row conditions only, selection conditions only, or all submitted conditions. An absent condition is never synthesized. Each branch starts from the original prefix and ends after one update. Business arguments, target ID, and prestate are held fixed. Replaying the all-submitted branch must reproduce the original return and state exactly.

This identifies only $T(w_i, (m_i, p_i^S))$ for the recorded proposal and condition subset S . It does not identify the outcome of an agent running under that subset: changed feedback would alter later actions and visited states. In particular, the 22 retries below exist because the recorded executor rejected earlier attempts. We report the first proposal of each episode separately from all proposals. Repeated local transitions are neither independent task samples nor new model calls. This is a retrospective analysis specified after inspecting the original results, not a preregistered policy comparison.

3. RECORDED PROTOCOLS AND INFORMATION BOUNDARIES

3.1. Mutable native-tool diagnostic (R9NT1)

The mutable-state cohort runs native ToolSandbox tools [6] at commit c8571d7. It uses state snapshots and a restricted dispatcher. Its 16 constructed cases cross four templates with four event regimes. Templates concern current-person messaging, literal-number messaging, reminders selected by description, and Wi-Fi dependencies. Regimes are stable state, relevant change before the episode, relevant change after a read, and unrelated change after a read. The inherited and verify-confirm arms share tools and materials; the latter adds generic pre-write resolution and post-write confirmation instructions. There are 32 primary and four separately reported repeated episodes.

One scheduled update may occur after the first qualifying read, including between sequential calls from one model

response. The read’s return is not altered. Skipped anchor reads are retained as unexposed, not rerun. All scheduled primary changes happened in the accepted batch: eight before an episode and 16 after reads. All primary post-read writes occurred in later model responses, not the same batch as the triggering read. This is cooperative scheduling, not a production concurrency frequency estimate.

3.2. Conditional SQLite diagnostic (R10)

The second cohort uses real file-backed SQLite with separate agent and environment connections. Twelve constructed cases concern service-to-configuration joins and highest-priority eligible-batch selection. Stable and irrelevant changes, protected-field changes, selection changes, and literal-ID controls are retained. A selection change can affect a route or competing candidate without changing the previously selected row. An optional selection token and optional row version are checked in the update transaction. All arms can use both capabilities.

Generic, minimal-update, intent-verification, and combined guidance are compared on the same 12 cases, giving 48 primary episodes with no extra repeats. The generic agent is not forced to rewrite old fields. Documentation explains omitted-field preservation, optional preconditions, and their scope. A reporting tool records the model’s completed/incomplete/uncertain assessment without consulting the evaluator. Forty scheduled updates occurred; eight episodes were stable. The raw and scored cohorts remain unchanged by our new analysis.

Both cohorts use native `tools/tool_calls` and linked `role=tool` returns, preserving required reasoning-history fields. Requests specify `deepseek-flash`, enabled thinking, high reasoning effort, a 16,384 output cap, and no temperature. Their per-episode caps are ten and eight model requests, respectively, with up to four sequential tools per response. Within-batch response labels and fingerprints match; this does not authenticate model weights or cross-batch identity. Tools affect only simulated data. No user simulator or model judge is used.

An earlier static ToolSandbox diagnostic (R7C1) achieved 14/14 per arm using a different text-action interface and pre-episode changes. It is context, not an extra control for these cohorts. Different tasks, interfaces, prompts, and enforcement capabilities prohibit attributing cross-cohort score differences to a single mechanism. Nor are the 32 or 48 method episodes independent domain samples.

4. RESULTS AND RETROSPECTIVE TRANSITION CHECKS

4.1. Live outcomes do not establish prompting gains

Table 1 reports only primary episodes. Both ToolSandbox arms finish 14/16 safely; the extra instruction uses more

Table 1. Recorded primary cohorts; rows from different cohorts are not a causal comparison. DB/tool calls exclude the SQLite reporting tool.

Cohort / arm	Safe	LLM calls	Tool calls
R9NT1 / inherited	14/16	58	52
R9NT1 / verify-confirm	14/16	66	59
R10 / generic	12/12	58	52
R10 / minimal	12/12	60	52
R10 / intent	12/12	58	54
R10 / both	12/12	61	56

model and tool calls without additional safe completions. In SQLite, all four arms finish 12/12. All episodes explicitly terminate. No final-goal/final-clean/trace-safe disagreement occurs in either primary cohort. The ToolSandbox tool-success proxy alone accepts four failing episodes. We do not present that deliberately weak proxy as the official benchmark evaluator.

In the four failing ToolSandbox primary episodes, a person-number or reminder-description binding changes after the initial query. Three episodes perform a post-write read. The message check reads a real message whose recipient identity differs from the target identity. The reminder checks read the old ID after the agent has restored its stale description onto that now-different object. Reading back one’s own written fields confirms neither current target selection nor preservation of the intervening update. These are behavior observations, not inferences from private reasoning text; the original final-state evaluator also detects the failures.

In SQLite, all 70 update attempts submit only the requested business field and at least one optional precondition. This includes generic guidance. Forty-eight updates commit and 22 are rejected, after which the agent re-queries and completes. A factorial assignment of instructions did not induce exclusive behavioral strategies, so equal scores establish neither prompt superiority nor statistical equivalence.

4.2. First proposals versus eventual completion

Table 2 reports 192 offline transitions for the first update of each of the 48 SQLite episodes. “Safe/unsafe” concerns the branch’s business effect. The remaining 88 transitions audit 22 later proposals. All 280 branches preserve their own original prefix; they are not connected into trajectories.

The original executor commits 26 first proposals, rejects 22, and ultimately completes all 48 episodes. Without the submitted selection predicates, eight business mutations address the old current target: four arms each on one routing and one ranking case. They are not eight independent task families. Row-only enforcement does not detect these changes because the old target row’s version remains unchanged. Importantly, the eight original actions include their protective

Table 2. First-proposal local replays ($n = 48$ per row), not live-agent outcomes. Only originally supplied predicates may be retained. Rejected columns classify the guard-free business patch.

Retained conditions	Committed		Rejected	
	safe	unsafe	safe patch	unsafe patch
None	40	8	0	0
Row only	26	8	14	0
Selection only	40	0	0	8
All submitted	26	0	14	8

predicates; none commits a wrong write in the recorded run.

The other 14 first proposals are rejected after an update to an owner or carrier advances the row version. Their actual patches touch only the requested max-attempts or ready-after field and preserve the changed business field. They are admissible in that saved prestate under this task specification. Thus all 22 rejections are genuine API conflicts, but only eight guard-free payloads cause local business harm. This distinction must not be generalized to other invariants, future mutations, or remote transaction semantics.

Across all 70 proposals, the four branches respectively yield (safe commits, unsafe commits, rejections) of (62,8,0), (48,8,14), (62,0,8), and (48,0,22). The 70 all-submitted branches exactly reproduce recorded results and states. This extends the earlier rejected-only inspection to every update and explicitly separates initial proposals from feedback-induced retries. It does not add model evidence or prove that a selection-only policy would finish 48 episodes safely.

4.3. Costs and statements remain separate

The mutable-tool cohort uses 124 primary requests with 367,260 input and 14,894 output tokens. Its 15 repeated-diagnostic requests are separate. SQLite uses 237 requests, 463,672 input and 49,679 output tokens. Reasoning tokens are already included in output; cached input is not added again. All amounts are usage, not verified billing. No new model request is made for the local replay.

In SQLite, combined guidance consumes 11.51% more total tokens than generic with the same final score. Different prompt lengths and action histories prevent a causal efficiency claim. Two audited completion explanations also contain incorrect details despite correct database outcomes. The frozen false-completion endpoint measures task completion, not sentence-level factuality, and remains zero; these textual observations do not replace a failed primary hypothesis with a new endpoint.

5. REPRODUCIBILITY, LIMITATIONS, AND NEXT EVIDENCE

The artifact fixes the source and original feedback hashes, verifies 1,091 feedback-manifest entries and 38 source files, and replays all 48 SQLite trajectories before branching. A separate public-task parser and business-state checker agree with native SQLite execution. Thirty-seven tests cover literal targets, ranking ties, protected fields, duplicate identities, condition subsets, changed inputs, and network blocking. Two full runs reproduce six scientific output files exactly; a fresh-package check is reported with the artifact. The analysis uses temporary databases and no credentials. It is one assistant’s independent implementation, not an independent human replication or formal proof.

The evidence has substantial limits. The tasks are explicitly constructed and narrow; no held-out public task distribution or second model family is tested. The after-read event rule conditions exposure on behavior. Prompt arms are not length matched. SQLite’s conditions are documented and callable by every arm; ToolSandbox has different write semantics. No cross-cohort gain is attributed to our code. Local replays condition on the actual guarded policy’s visited states and do not measure alternative-policy recovery, latency, or safety. In particular, counting a legal rejected compare-and-set as an agent failure would invalidate the analysis.

Runtime enforcement and version-versus-decision distinctions already have strong precedents [2, 4, 5]. Our artifact is a diagnostic protocol and case study, not a novel guard or proof that conditional rejection audits are new. A publishable extension must demonstrate incremental value on independent tasks: for example, whether full conditional-action semantics materially change conclusions drawn from outcome-only or rejection-count evaluations. That requires independently checked task invariants, same-capability controls, and live closed-loop validation; repeating these saturated prompt arms is not sufficient.

6. CONCLUSION

Recorded success can depend on a conditional tool contract, and recorded rejection can be legitimate control flow. Post-write reads may check a stale identity rather than the user’s target, while a protective predicate can reject a payload the agent could not safely apply unconditionally. Our two cohorts and exhaustive saved-proposal replays make these boundaries explicit without claiming new live failures, a better controller, or general safety. The current result supports evidence consolidation and a narrower external validation question, not additional same-template sampling.

7. ACKNOWLEDGMENT AND ARTIFACT STATUS

ChatGPT assisted with code, tests, analysis, literature checking, and drafting. Human authorship, independent review, and final submission checks remain pending. The inherited `spconf` style has not been byte-authenticated against a current official distribution. Original research records and earlier manuscripts remain unchanged.

8. REFERENCES

- [1] R. Fielding, M. Nottingham, and J. Reschke, “HTTP semantics,” RFC 9110, 2022, Sec. 13.
- [2] H. Wang, C. M. Poskitt, and J. Sun, “AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents,” arXiv:2503.18666v3, 2025.
- [3] D. Lilienthal and S. Hong, “Mind the gap: Time-of-check to time-of-use vulnerabilities in LLM-enabled agents,” arXiv:2508.17155, 2025.
- [4] V. Reddy, S. R. Challaram, and A. Basu, “Reason less, verify more: Deterministic gates recover a silent policy-violation failure mode in tool-using LLM agents,” arXiv:2607.07405v2, 2026.
- [5] Y. Lyu, Y. Ren, R. Lai, and W. Liu, “From version conflicts to decision conflicts: Selective revalidation for long-running AI agents,” arXiv:2609.08015v1, 2026.
- [6] J. Lu et al., “ToolSandbox: A stateful, conversational, interactive evaluation benchmark for LLM tool use capabilities,” arXiv:2408.04682, 2024.